24. Trees, Heaps, and Priority Queues

24. Trees, Heaps, and Priority Queues

Phase: Phase 4 — Algorithms and data structures
Estimated time: 2 days
Slug: `24-trees-heaps`

Learning objectives

- Implement binary tree traversals
- Use heap.Interface for priority queues
- Know BST search costs
- Apply heaps to scheduling problems

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Trees and heaps power priority queues, schedulers, and hierarchical configs. Binary heaps give cheap extract-min.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Binary tree: left/right children; traverse inorder/preorder/postorder.
- Heap property: parent ≤ children (min-heap) or ≥ (max-heap).
- container/heap requires Push/Pop methods on a type.
- Priority queues schedule work by urgency.
- Balanced trees (overview) keep operations logarithmic.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-trees, && cd /tmp/gw-trees,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "container/heap"
    "fmt"
)

type IntHeap []int

func (h IntHeap) Len() int           { return len(h) }
func (h IntHeap) Less(i, j int) bool  { return h[i] < h[j] }
func (h IntHeap) Swap(i, j int)       { h[i], h[j] = h[j], h[i] }
func (h *IntHeap) Push(x any)         { *h = append(*h, x.(int)) }
func (h *IntHeap) Pop() any {
    old := *h
    n := len(old)
    x := old[n-1]
    *h = old[:n-1]
    return x
}

func main() {
    h := &IntHeap{3, 1, 2}
    heap.Init(h)
    fmt.Println(heap.Pop(h))
}
```

Run it:

```
go run .
```

Expected output:

```
1
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Init a min-heap and pop in order.
2. Implement a binary tree Insert for ints.
3. Traverse and collect values.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Binary tree

```
type Node struct {
    Val int
```

```
    Left, Right *Node
}
```

Traversals: inorder, preorder, postorder, level-order (BFS with queue).

Heaps / priority queues

`container/heap` needs `Len`, `Less`, `Swap`, `Push`, `Pop`.

Heaps give O(log n) push/pop and O(1) peek — perfect for: - scheduling jobs by priority - Dijkstra's algorithm - top-K queries

BST intuition

Balanced BST: O(log n) search. Unbalanced (sorted inserts): O(n). Production often uses maps (hash) or specialized trees.

Failure modes to watch while coding

- Assuming BST is always log n
- Incorrect Less making heap invalid
- Mutating heap slice outside heap functions

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- heapq ≈ container/heap with more ceremony in Go.
- dataclasses for nodes vs structs.
- Recursion for traversal is fine for shallow trees.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

For scheduling, store (priority, sequence, item) to keep ties deterministic.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Trees, Heaps, and Priority Queues" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Assuming BST is always log n
- Incorrect Less making heap invalid
- Mutating heap slice outside heap functions

Exercises

Work in `exercises/24-trees-heaps/`. Read that folder's `README.md` before you code.

What you will build

Implement heap/tree helpers so extract-min / traversal tests pass.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/24-trees-heaps/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Complexity of heap push?
2. Name three traversals
3. When use priority queue in infra?

Further reading

- https://pkg.go.dev/container/heap